PYGERS-MEG USER GROUP

Pygers-MEG-to-BIDS

One script turns a session of raw OPM-MEG, eye-tracking, MRI and behavioral data into a validated BIDS dataset, with an HTML report for every step.

```
bash run_bids-pipeline.sh sub-01.yaml
```

November 2026 · github.com/mrribbits/pygers-meg

What the pipeline does

IN

One raw session

- Cerca .cMEG recordings
- EinScan head shape
- EyeLink .edf
- MRI DICOMs
- PsychoPy .csv logs

RUNS

Six notebooks, one Slurm job

Settings come from YAML files, so nobody edits the notebooks. Run by hand, the notebooks still work exactly as before.

OUT

A BIDS dataset

- Anonymized, validated
- MEG, eye tracking, MRI, behavior
- An HTML report per notebook
- A draft Methods text

The six notebooks it runs

#	Notebook	What it does
01	01_Explore_Raw_OPM-MEG_Files	cMEG → FIF, screens for bad channels
02	02_Explore_Eye_Data_Files	EyeLink .edf → BIDS with eye2bids
03	03_Dicom_to_BIDS_heudiconv	MRI DICOMs → NIfTI, defaced copies
04	04_FIF_to_BIDS	MEG, MRI and behavior into BIDS
05	05_Sync_eye_to_MEG	Aligns eye tracking to the MEG clock
06	06_Validate_BIDS	BIDS validator, Methods text

All in 01-explore-raw-files/. The pipeline runs these exact files; copies such as my_04.ipynb are ignored.

Settings in three levels

FACILITY

`facility.yaml`

The MEG lab itself: hardware, institution, scanner time zone. Lives in the repository; the same for every study.

STUDY

`global-study.yaml`
`processing.yaml`
`private.yaml · text/`

Paths, your software, what to run, how to process, the dataset's description, DAYSBACK. Set once per study.

SUBJECT

`subjects/sub-01.yaml`

One session: labels, recording prefixes, EinScan files, bad channels, participant details. The file you submit.

A study changes a facility value under facility_overrides: in global-study.yaml, never in facility.yaml.

Your study folder

```
<study>/
├── bids-config/              your settings
├── bids/                     anonymized BIDS only
└── sourcedata/
    ├── code/heuristic.py
    └── sub-01/ses-01/
        ├── raw/              cMEG einscan eye mri behav
        ├── fif/
        ├── eye-staging/
        ├── mri-staging/
        ├── defaced_anat/
        └── reports/
```

raw/ holds the files from the lab, as they came. Nothing changes them.

sourcedata/ sits beside bids/, never inside it: it can identify the participant.

One participant per visit? Set sessions: false and the ses-01/ level disappears.

Where it lives in the repository

```
01-explore-raw-files/
 ├──── 01_…ipynb … 06_…ipynb
 ├──── run_bids-pipeline.sh          checks, then submits the Slurm job
 └──── bids-pipeline/
     ├──── run_bids-pipeline.py      runs the notebooks, writes the reports
     ├──── bids_config.py            reads and checks the YAML files
     ├──── bids_config.schema.yaml   the rules for every setting
     ├──── facility.yaml             the lab's settings
     ├──── templates/bids-config/    the folder you copy for a study
     └──── tests/                    checks for the pipeline's own code
```

Two READMEs: **templates/bids-config/README.md** to set up and run a study, **bids-pipeline/README.md** for how it works inside.

Before you start: your software

New environment

```
conda env create \
-f scotty-environment.yml \
-n pygers-meg
```

Already has papermill, cmeg2fif and the fixed eye2bids fork.

Made before Oct 27, 2026

```
pip install "papermill==2.7.0" …
pip install --force-reinstall \
  --no-deps "eye2bids @ …@0991c4c…"
cmeg2fif --version   # needs 2.16
pip install --upgrade \
  "cmeg2fif @ …@04845fc…"
```

Full lines in the speaker notes. Then restart Jupyter kernels.

Yours to keep: the HeuDiConv, pydeface and BIDS validator containers, and edf2asc. Their paths go in the software: section of global-study.yaml.

Setting up a study, in four steps

1

Copy the template

templates/bids-config/ → <study>/bids-config/, outside the BIDS dataset.

2

Edit the files

Every setting has a comment. One subject file per session (copy sub-01.yaml).

3

Check

Two checks, neither changes anything. They list every problem at once.

4

Run a session

Submit the subject file. Open index.html in reports/ when it’s done.

Two kinds of placeholders

`[yourpath]`

Fill in before the first run

Data paths and your containers, plus DAYSBACK in private.yaml. The pipeline won’t run until they’re set.

`EDIT ME`

Can wait until you share

In the dataset description (end of global-study.yaml) and text/: authors, release date, license text. The pipeline runs and lists them as reminders.

Filled them in later? Run notebook 04 again with write_dataset_files: true, then 06.

Two checks before you submit

```
python bids-pipeline/bids_config.py sub-01.yaml
```

While you edit, quick. The YAML files: misspelled or missing settings, wrong values, settings that disagree, and whether the .edf and DICOMs can be found. Add `--show 04` to print what notebook 04 would receive.

```
bash run_bids-pipeline.sh sub-01.yaml --check
```

Before you submit. All of that, plus what a real run needs: the raw recordings, cmeg2fif, edf2asc, each container actually starting, and which notebooks would run.

Run both from 01-explore-raw-files, with `conda activate pygers-meg` first.

Running a session

```
conda activate pygers-meg
cd <your pygers-meg>/01-explore-raw-files
bash run_bids-pipeline.sh /jukebox/<lab>/<study>/bids-config/subjects/sub-01.yaml
```

Option	What it does
(none)	Checks everything, then submits one Slurm job
--check	Only checks; submits nothing
--steps 04,05	Only these notebooks (plus the step rules)

Time, memory, CPUs and optional email come from the slurm: section of global-study.yaml.

Bad channels: the recommended way

1. **Run 01 first, on its own.** --steps 01

2. **Look at its HTML report**, section 6.1: the candidates (flat, flat sensor, noisy, marked bad in Cerca) and when each one was flat or noisy.

3. **Set bad_channels: in the subject file:** a list, auto, or auto with add / keep.

4. **Run the rest.** --steps 02,03,04,05; 01 doesn't need to run again.

You can always change your mind later: update the subject file and run --steps 04,05.

What happens in a run

1 · PRE-FLIGHT

Config, raw files, software, containers. Any problem stops it here, before anything changes.

2 · PLAN

The steps you chose, minus data the study doesn’t have.

3 · RUN

Each notebook with papermill, reading the run’s settings.yaml.

4 · LOCK

03–06 wait their turn, so two jobs never write the dataset at once.

5 · REPORTS

HTML per notebook and an index page with the summary.

Which notebooks run

1. **Always in order, 01 → 06.** --steps 05,04 still runs 04 first.
2. **Only the ones you choose.** From steps: in global-study.yaml, or --steps, which takes its place.
3. **06 is always added** and runs last.
4. **No data, no notebook.** data.eye_tracking: false skips 02 and 05; data.mri: false skips 03, even if chosen.
5. **04 without 05 gives a warning** (studies with eye tracking). 05 isn't added for you.

What each notebook needs from others

Notebook	Needs	Made by
04	The task and empty-room FIF files	01
04	The bad-channel candidates (with bad_channels: auto)	01
04	The converted MRI (studies with MRI)	03
05	The eye2bids files	02
05	The MEG recording in the BIDS dataset	04
06	A BIDS dataset (dataset_description.json)	04, any subject

If the notebook that makes them isn’t running and the files aren’t there, the run stops before anything changes. Except 06: it’s skipped, with a warning.

During a run, and between jobs

DURING A RUN

A notebook that fails or is skipped skips the ones that need its files: 01 fails, so 04 and 05 are skipped.

Except 06: it still runs, because it checks the whole dataset.

OUT-OF-DATE FILES

Warns, and the run goes on, when a notebook that isn't running has input newer than its output:

04 without 01: raw cMEG or EinScan files newer than the FIFs.

04 without 03: DICOMs newer than the converted MRI.

05 without 02: the .edf newer than the eye2bids files.

BETWEEN JOBS

Notebooks 03–06 take a lock on the dataset, so two subjects' jobs never write it at the same time.

The other job waits its turn; 01 and 02 never wait.

What you get back

```
sourcedata/sub-01/ses-01/reports/
├── latest → 20261027-134456
├── slurm-<job id>.log
└── 20261027-134456/
    ├── index.html          start here
    ├── 01_….html … 06_….html
    ├── 01_….ipynb … 06_….ipynb
    ├── settings.yaml
    ├── config/
    └── pipeline.log
```

index.html: each notebook's status, run time, why one failed or was skipped, and the validator's counts.

Each report: text, tables and figures; a button shows the code. A failed cell always shows its code.

settings.yaml, config/: exactly what this run used.

A first session, start to finish

#	Run	Then
1	run_bids-pipeline.sh … --check	Fix whatever it lists
2	run_bids-pipeline.sh … --steps 01	Read 01’s report; set bad_channels in the subject file
3	run_bids-pipeline.sh …	All six notebooks; open index.html
4	(read notebook 06’s report)	Validator findings, placeholders, Methods text
5	Next subject	Copy sub-01.yaml, edit it, submit it

bad_channels can be a list, auto (01’s candidates), or auto with add/keep changes.

Notebook 06 closes the loop

BIDS validator

Checks the whole dataset, read-only. Errors first, then a table of which notebook fixes what.

Placeholders left

Every EDIT ME that reached the dataset, which the validator can’t see.

A Methods text

Like fMRIPrep’s boilerplate: a draft with versions and references, read from the dataset itself.

`bids/derivatives/bids-conversion/group_desc-methods.md`

Questions?

Set up and run a study: 01-explore-raw-files/bids-pipeline/templates/bids-config/README.md

How it works inside: 01-explore-raw-files/bids-pipeline/README.md

Repository: github.com/mrribbits/pygers-meg